

Mathematical Calculator

A Modular, Menu-Driven Console Application in Python

Submitted by	Abhinav A D
Registration Number	26BCE11228
Project Title	Mathematical Calculator
Programming Language	Python 3
Programme	VITyarthi - Build Your Own Project (Flipped Course Evaluation)
Submission Date	September 2026

Detailed project report covering the problem statement, requirements, architecture, design diagrams, implementation, results, testing, challenges, learnings and future enhancements.

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement and Objectives
4. Functional Requirements
5. Non-Functional Requirements
6. System Architecture
7. Design Diagrams (Use Case, Workflow, Sequence, Class/Component, ER)
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Mathematics is used in almost every field of study, and students routinely need quick, reliable answers to small problems: the average of a data set, the product of two matrices, the value of a polynomial at a point, the volume of a cylinder, or the greatest common divisor of two numbers. General-purpose calculators can do some of these, but they rarely bring statistics, linear algebra, algebra, geometry and number theory together in one place with clear, step-by-step prompts.

Mathematical Calculator is a menu-driven console application written in Python that combines five commonly needed mathematical tools into a single program. The user chooses a module from a main menu, is prompted for the required inputs, and receives the computed result immediately. After every calculation the program returns to the main menu, so many different calculations can be performed in one session.

The project applies core programming concepts: functions and modular design, control flow, loops, lists and nested lists, recursion, string formatting, the standard library (`math`, `collections.Counter`), and input validation. Each mathematical tool lives in its own Python file and is imported by a small controller, `main.py`.

Project at a glance

Item	Details
Application type	Menu-driven console (command-line) application
Language / runtime	Python 3 (no third-party packages required)
Number of source files	6 (<code>main.py</code> , <code>statistics.py</code> , <code>matrix.py</code> , <code>polynomial.py</code> , <code>geometrical.py</code> , <code>gcd_and_lcm.py</code>)
Functional modules	5 (Statistics, Matrix, Polynomial, Geometry, GCD & LCM)
Persistent storage	None (all data is held in memory for the duration of a calculation)

3. Problem Statement and Objectives

3.1 Problem statement

Students and beginners who need to perform routine mathematical computations across different topics typically have to switch between several tools: a scientific calculator for arithmetic, a spreadsheet for statistics, separate software for matrices, and manual formulas for geometry or number theory. Manual calculation is slow and error-prone (for example, expanding a 3x3 determinant by hand or finding the LCM of large numbers), and many tools give only the final answer without a guided, transparent input process.

There is a need for a single, lightweight, easy-to-run program that offers several common mathematical computations through a consistent, guided interface, without installing any external software or libraries.

3.2 Scope

The project covers descriptive statistics, basic matrix operations, polynomial evaluation, area/perimeter/surface-area/volume calculations for six standard shapes, and GCD/LCM of two integers. It is a console program for a single user; it does not include a graphical interface, symbolic algebra, or data storage.

3.3 Target users

School and college students, teachers demonstrating concepts, and any user who needs quick verification of hand-worked mathematical results.

3.4 Objectives

- Provide **five independent mathematical modules** accessible from a single main menu.
- Design a **clear input/output structure**: every module prompts for its inputs, validates them where required, and prints labelled results.
- Implement the mathematics using **correct, well-known algorithms** (Euclidean algorithm, Horner's method, cofactor expansion, etc.).
- Keep the code **modular, readable and maintainable**, with one responsibility per file.
- Keep the program **lightweight**: standard library only, instant responses, no external dependencies.
- Document the design and verify correctness through a documented set of validation tests.

4. Functional Requirements

The system provides five major functional modules (the guideline minimum is three), a clear input/output structure and a logical user workflow. The requirements are listed below with identifiers used later in the testing section.

ID	Module	Requirement	Inputs	Outputs
FR-1	Main Menu	Display a banner and the list of modules, read the user's choice, call the chosen module, and return to the menu until the user exits. Reject invalid choices with a message.	Menu choice (1-6)	Menu, messages
FR-2	Statistics	Compute descriptive statistics of a data set of real numbers.	Space-separated numbers	Mean, median, mode, variance, standard deviation
FR-3	Matrix	Perform addition, subtraction, multiplication, transpose and determinant of user-entered matrices, checking dimension compatibility.	Operation choice; rows, columns and entries of one or two matrices	Result matrix or determinant, or an explanatory error message
FR-4	Polynomial	Evaluate a polynomial of any degree at a given value of x .	Degree, coefficients (highest power first), x	$P(x)$
FR-5	Geometry	Compute measurements for six shapes: Circle, Rectangle, Triangle, Square, Sphere, Cylinder.	Shape choice; dimensions (radius, length, width, base, height, side)	Area, perimeter/circumference, surface area, volume as applicable
FR-6	GCD & LCM	Compute the greatest common divisor and least common multiple of two integers, including zero and negative values.	Two integers	GCD, LCM
FR-7	Session	Allow repeated calculations in one session and a clean exit.	Choice 6	Thank-you message

4.1 Detailed behaviour of each module

- **Statistics:** the mean is the sum divided by n ; the median is the middle value (or the average of the two middle values for even n) of the sorted data; the mode is the value(s) with the highest frequency, or "No mode" if all values are unique; variance is the population variance (sum of squared deviations divided by n); standard deviation is its square root.
- **Matrix:** supports Addition, Subtraction, Multiplication, Transpose and Determinant. Each row is validated to contain exactly the declared number of columns; operations check that dimensions are compatible before computing.
- **Polynomial:** coefficients are read from the highest power down to the constant term and the value is computed using Horner's method.
- **Geometry:** Circle (area, circumference), Rectangle (area, perimeter), Triangle (area from base and height), Square (area, perimeter), Sphere (surface area, volume), Cylinder (total surface area, volume).
- **GCD & LCM:** GCD by the Euclidean algorithm; $LCM = |a \times b| / GCD(a, b)$, returning 0 when either number is 0.

5. Non-Functional Requirements

Six non-functional requirements (the guideline minimum is four) were specified for the project, together with how each is met in the implementation.

ID	Category	Requirement	How it is achieved
NFR-1	Usability	The interface must be easy for a first-time user; every prompt states exactly what to enter.	Numbered menus, descriptive prompts (e.g. "Enter number of rows for Matrix A:"), labelled output, and automatic return to the main menu after each task.
NFR-2	Maintainability / Modularity	Each mathematical area must be independently modifiable without affecting the others.	One file per module, one public entry function per module, and a thin controller (main.py). Helper logic is kept as nested functions close to where it is used.
NFR-3	Reliability / Correctness	The same input must always give the same, mathematically correct output.	Standard algorithms (Euclid, Horner, cofactor expansion) and deterministic code with no random or external state. Verified with 17 validation tests.
NFR-4	Error handling	Invalid menu options, incompatible matrix sizes, malformed rows and empty data must be handled gracefully.	Explicit checks: invalid-choice messages, dimension checks before add/subtract/multiply, square-matrix check for determinant, row-length re-prompt loop, empty-data check.
NFR-5	Performance / Resource efficiency	Results must appear instantly for typical classroom-sized inputs and use negligible memory.	Linear or near-linear algorithms for statistics and polynomials, $O(\log n)$ GCD, and in-memory lists only; no files, network or databases.
NFR-6	Portability / Low dependencies	The program must run on any machine with Python 3 without installation steps.	Uses only the standard library (math, collections); no pip packages; works on Windows, macOS and Linux terminals.

Note on security: the application is an offline, single-user calculator with no authentication, network access or stored data, so security requirements such as access control do not apply. The only security-relevant design choice is that user input is converted with `int()` / `float()` and never passed to `eval()` or `exec()`, so arbitrary code cannot be injected through the prompts.

6. System Architecture

The system follows a simple **two-tier modular architecture**: a control layer (`main.py`) that owns the menu loop and routing, and a module layer of five independent calculators. Every module is self-contained: it reads its own input, validates, computes and prints. Modules do not depend on one another, which means any module can be added, replaced or tested in isolation.

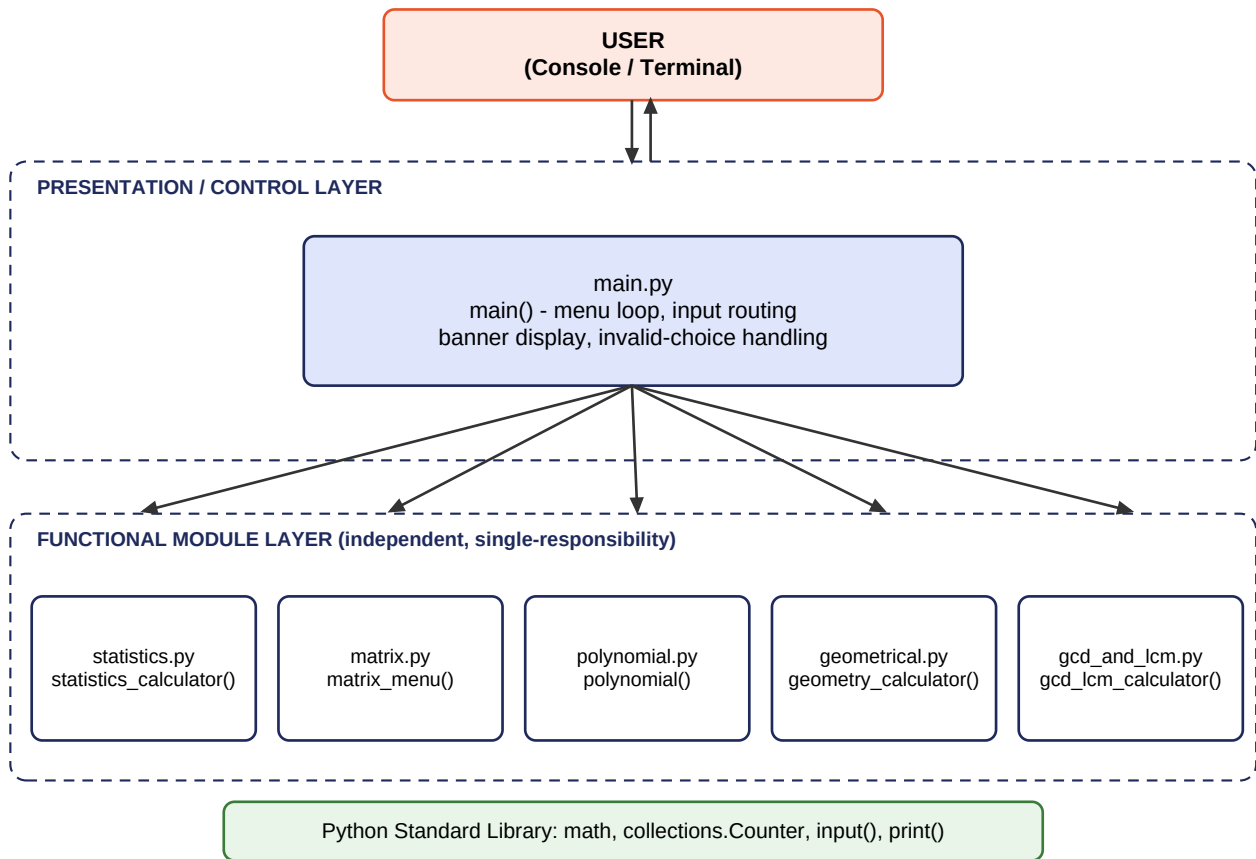


Figure 1: Layered system architecture

6.1 Project folder structure

```

mathematical-calculator/
|-- main.py           # Entry point: menu loop and routing
|-- statistics.py     # Statistics calculator (mean, median, mode, variance, SD)
|-- matrix.py         # Matrix calculator (add, subtract, multiply, transpose, det)
|-- polynomial.py     # Polynomial evaluator (Horner's method)
|-- geometrical.py    # Geometry calculator (6 shapes)
|-- gcd_and_lcm.py    # GCD and LCM calculator (Euclidean algorithm)
|-- README.md         # Overview, features, install/run/test instructions
`-- statement.md      # Problem statement, scope, target users, features
  
```

6.2 Technologies and tools

Tool	Purpose
Python 3	Implementation language
math module	pi and square root for geometry and standard deviation
collections.Counter	Frequency counting for the mode
Git and GitHub	Version control and repository submission
Any terminal / IDLE / VS Code	Running the program

7. Design Diagrams

7.1 Use Case Diagram

The only actor is the **User**. Each module exposes one main use case; "Select module" and "Exit" belong to the main menu.

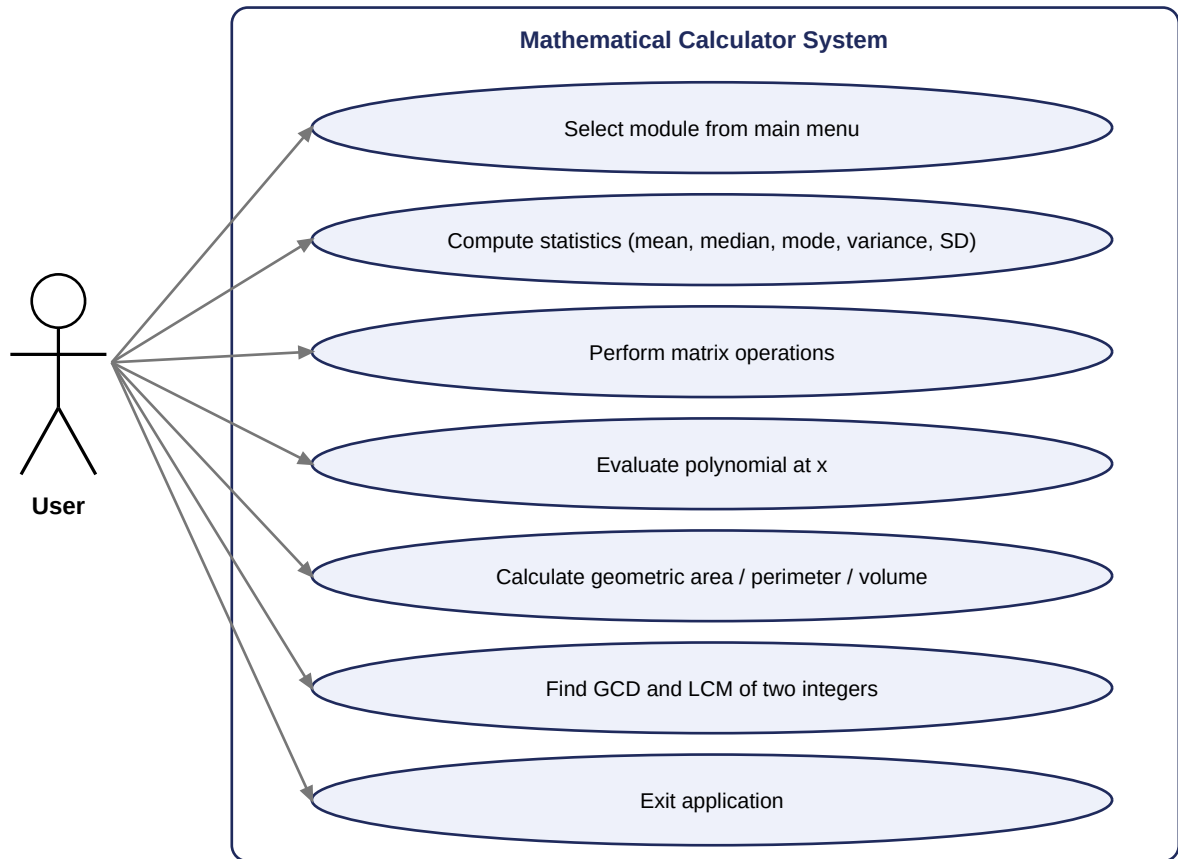


Figure 2: Use case diagram

7.2 Workflow Diagram

The overall workflow is a loop: the menu is shown, a choice is read and validated, the selected module runs to completion (read inputs, validate, compute, display), and control returns to the menu until the user selects Exit.

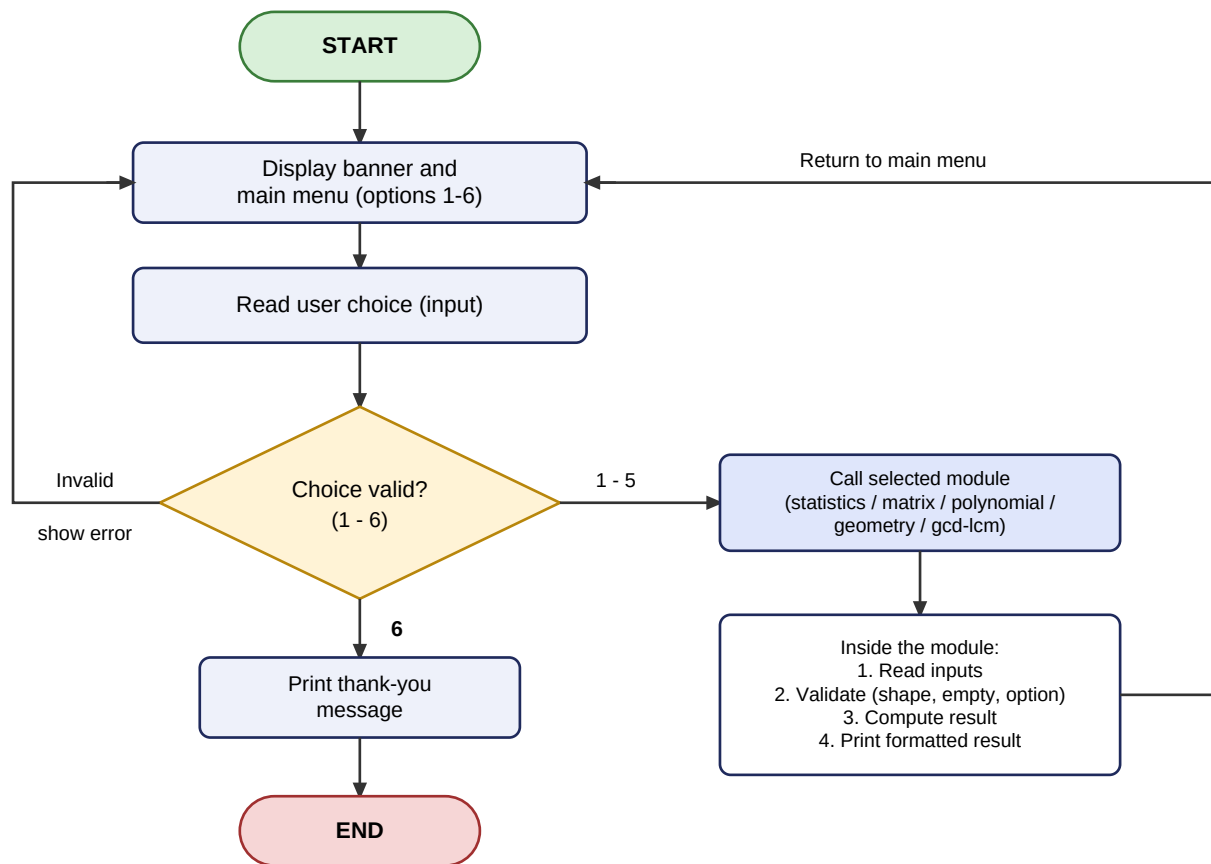


Figure 3: Main workflow of the application

7.3 Sequence Diagram

Figure 4 shows a complete session in which the user runs the Statistics calculator once and then exits. The interaction pattern is identical for the other four modules, only the module participant and its computations change.

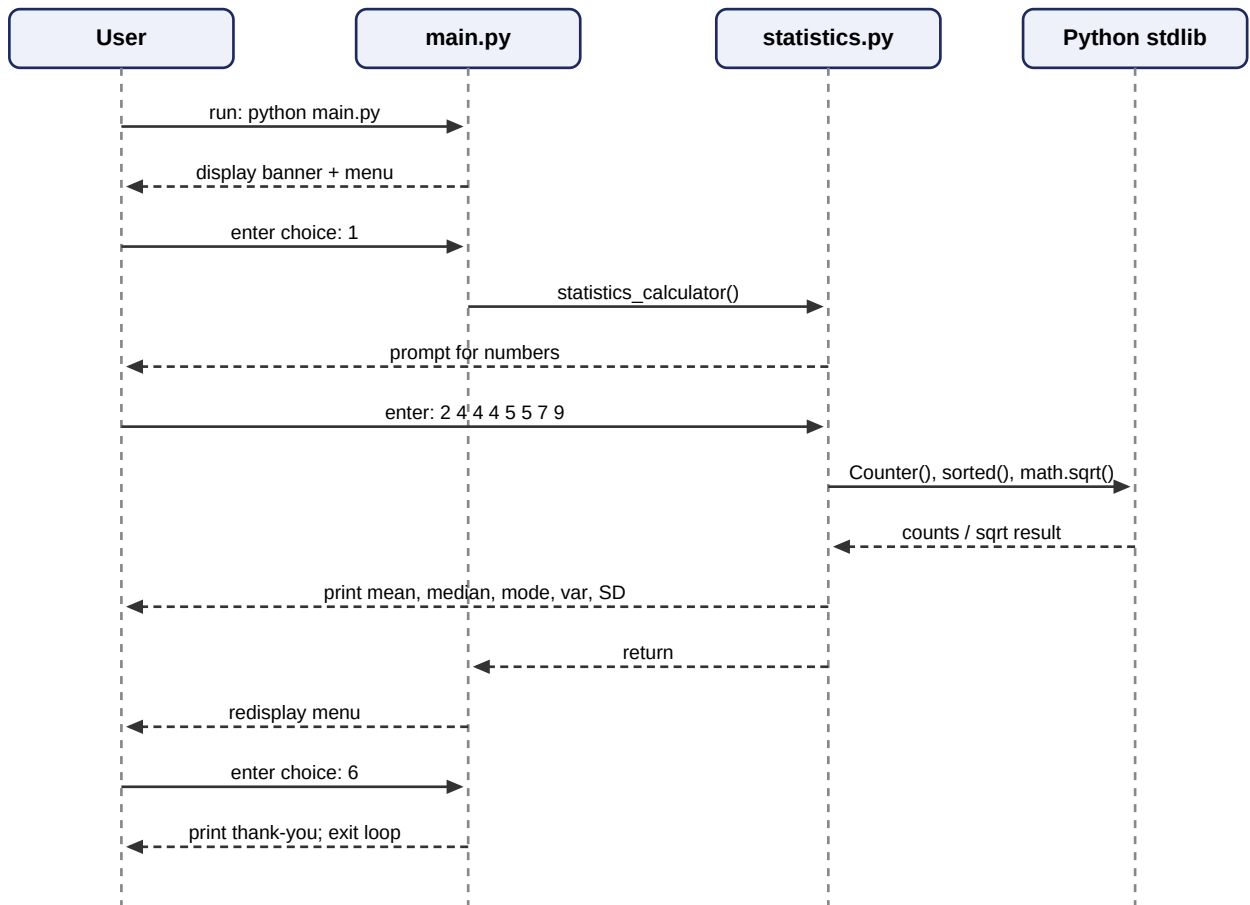


Figure 4: Sequence diagram for a Statistics session

7.4 Class / Component Diagram

The program is procedural rather than object-oriented, so a **component diagram** is used. Each Python file is a component exposing one public function; helper functions (marked with "-") are nested inside the entry function and are private to it. `main.py` depends on all five modules.

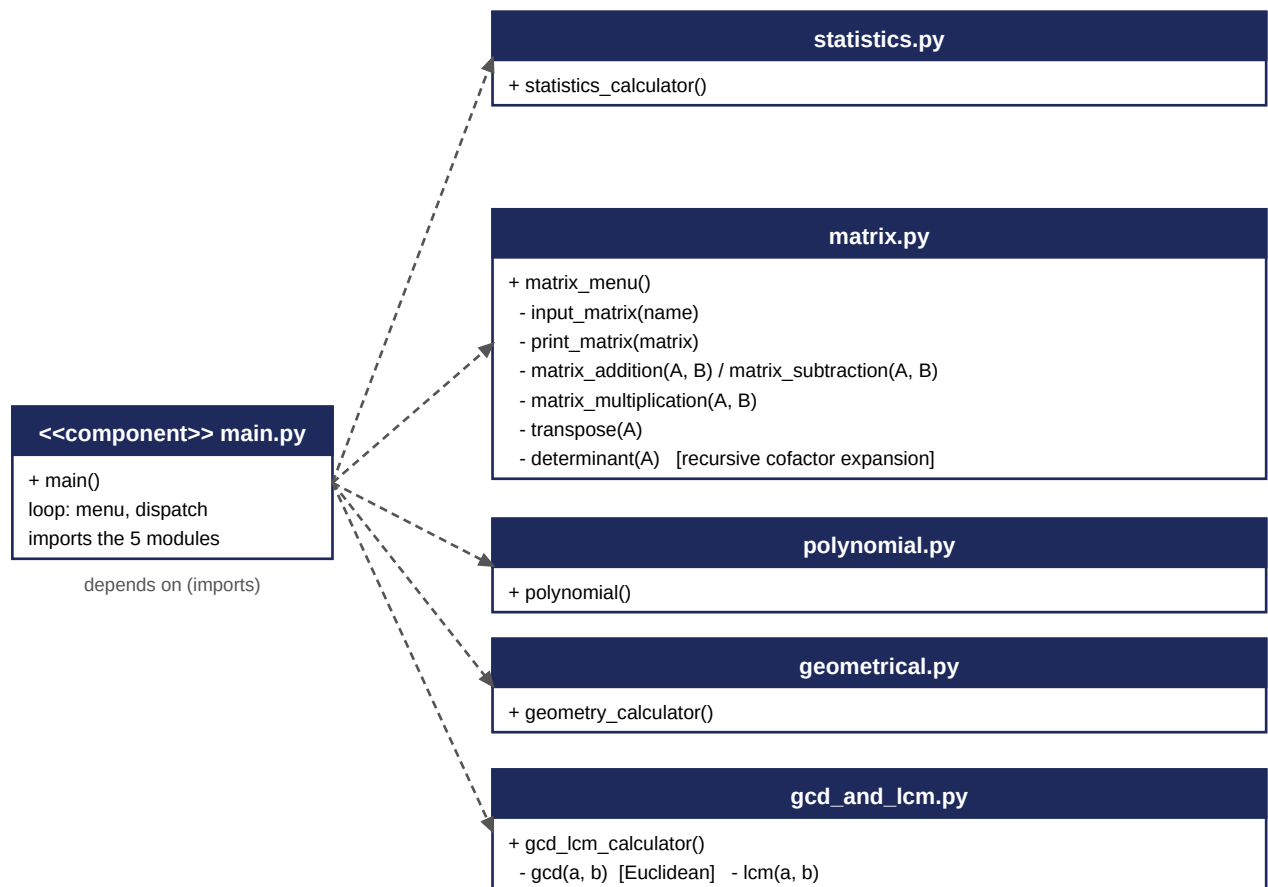


Figure 5: Component diagram (files and their functions)

7.5 ER Diagram / Storage Design

The application uses **no persistent storage** (no files or database), so an ER diagram and relational schema are not applicable. All data exist only in memory during a calculation. The in-memory data structures used are summarised below in place of a schema.

Module	Data structure	Description
Statistics	list of float; Counter	Input numbers; frequency table used to find the mode
Matrix	list of lists of float	Row-major matrices; result matrices are built as new lists
Polynomial	list of float	Coefficients from highest power to constant
Geometry	float variables	Dimensions and computed measurements
GCD & LCM	int variables	Two integers and the computed results

8. Design Decisions and Rationale

Decision	Rationale
One file per module with a single entry function	Keeps responsibilities separate, makes each calculator easy to read, test and extend, and lets main.py stay very small.
Menu-driven console interface in an infinite loop	Simple for beginners, works everywhere, and allows many calculations in a single session without restarting.
Helper functions nested inside the module entry function (e.g. gcd/lcm inside gcd_lcm_calculator, all matrix helpers inside matrix_menu)	Keeps helpers private to their module, avoids polluting the global namespace, and groups the logic that belongs together.
Euclidean algorithm for GCD, and $\text{LCM} = a*b / \text{GCD}$	Efficient $O(\log n)$ method; using the absolute value handles negative inputs; a zero check avoids division by zero.
Horner's method for polynomial evaluation	Needs only n multiplications and n additions, avoids computing large powers separately, and is numerically more stable.
Recursive cofactor (Laplace) expansion for determinant	Directly follows the textbook definition, works for any square size, and is easy to verify. Its factorial cost is acceptable for classroom-sized matrices.
Matrices stored as lists of lists of floats; output formatted with :g	Natural Python representation; the :g format prints 19.0 as 19 and keeps output clean while still showing decimals when needed.
Row-length validation with a re-prompt loop when reading matrices	Prevents ragged matrices, which would cause index errors later, without forcing the user to restart.
Dimension checks before matrix operations	Reports mathematically invalid requests (e.g. adding a 2x2 to a 3x3) with a clear message instead of a crash.
Population variance (divide by n) and standard deviation	Treats the entered numbers as the complete data set, which is the common convention for classroom exercises.
Mode returns a list, or "No mode" if all values occur once	Correctly handles multimodal data and avoids reporting a meaningless mode.
Standard library only	No installation steps; minimal resource use; portable across platforms.

9. Implementation Details

This section explains how each file works. Complexity is given in terms of the input size n (or matrix dimension n).

9.1 main.py - controller

`main()` runs a `while True` loop that prints the banner (built by repeating "." and "~" 40 times) and the numbered menu, reads the choice, and dispatches to the matching function using an `if/elif` chain. Choice 6 prints a farewell message and breaks the loop; any other value prints "Invalid choice. Please try again." and the menu is shown again. The `if __name__ == "__main__":` guard makes the file safe to import.

9.2 statistics.py - Statistics Calculator

The numbers are read as one line, split on whitespace and converted to floats. The median sorts the data ($O(n \log n)$) and picks the middle element(s). The mode uses `Counter` to count occurrences in one pass and reports every value with the maximum count. Variance and standard deviation use the mean computed earlier.

```
mean = sum(data) / n
counts = Counter(data)
max_count = max(counts.values())
mode = "No mode" if max_count == 1 else [x for x, c in counts.items() if c == max_count]
variance = sum((x - mean) ** 2 for x in data) / n
standard_deviation = math.sqrt(variance)
```

9.3 matrix.py - Matrix Calculator

`input_matrix()` reads the row and column counts, then reads each row, re-prompting until exactly the right number of values is entered. Addition and subtraction use nested list comprehensions ($O(\text{rows} \times \text{cols})$). Multiplication uses the standard triple loop ($O(n^3)$) after checking that the columns of A equal the rows of B . Transpose uses `zip(*A)`. The determinant is computed recursively by cofactor expansion along the first row; base cases are 1×1 and 2×2 matrices.

```
def determinant(A):
    n = len(A)
    if n == 1: return A[0][0]
    if n == 2: return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    result = 0
    for col in range(n):
        minor = [[A[i][j] for j in range(n) if j != col] for i in range(1, n)]
        result += ((-1) ** col) * A[0][col] * determinant(minor)
    return result
```

9.4 polynomial.py - Polynomial Evaluator

The user gives the degree and then each coefficient from x^n down to the constant. Horner's method rewrites $a_n x^n + \dots + a_0$ as $(\dots((a_n x + a_{n-1})x + a_{n-2})x + \dots) + a_0$, giving $O(n)$ time.

```
result = 0
for coefficient in coefficients:
    result = result * x + coefficient
```

9.5 geometrical.py - Geometry Calculator

A shape menu selects one of six branches. The formulas used are listed below.

Shape	Formulas
Circle	Area = πr^2 ; Circumference = $2\pi r$

Shape	Formulas
Rectangle	Area = $l \times w$; Perimeter = $2(l + w)$
Triangle	Area = $(1/2) \times \text{base} \times \text{height}$
Square	Area = s^2 ; Perimeter = $4s$
Sphere	Surface area = $4 \pi r^2$; Volume = $(4/3) \pi r^3$
Cylinder	Surface area = $2 \pi r (r + h)$; Volume = $\pi r^2 h$

9.6 gcd_and_lcm.py - GCD and LCM Calculator

The Euclidean algorithm repeatedly replaces (a, b) with $(b, a \bmod b)$ until b is zero; the absolute value of the remaining a is the GCD, so negative inputs are handled. The LCM is $|a \times b|$ divided by the GCD, and it returns 0 if either number is 0 (avoiding division by zero).

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return abs(a)

def lcm(a, b):
    if a == 0 or b == 0:
        return 0
    return abs(a * b) // gcd(a, b)
```

9.7 Complexity summary and version control

Module	Time complexity	Notes
Statistics	$O(n \log n)$	Dominated by sorting for the median
Matrix add / subtract / transpose	$O(r \times c)$	Single pass over all entries
Matrix multiply	$O(r \times c \times k)$	Triple loop; $O(n^3)$ for square matrices
Determinant	$O(n!)$	Cofactor expansion; fine for small n
Polynomial	$O(n)$	Horner's method
GCD	$O(\log \min(a, b))$	Euclidean algorithm

The source is managed with Git and published in a GitHub repository together with README.md (title, overview, features, technologies, install/run and testing steps) and statement.md (problem statement, scope, target users and high-level features), as required by the submission guidelines. To run the program: install Python 3, open a terminal in the project folder and run `python main.py`.

10. Screenshots / Results

The following terminal captures were produced by running the actual project code. Text after each prompt is the value typed by the user.

```
● ● ● python main.py - Main menu

.....
-----
      MATHEMATICAL CALCULATOR
-----
.....

1. Statistics Calculator
2. Matrix Calculator
3. Polynomial Calculator
4. Geometry Calculator
5. GCD & LCM Calculator
6. Exit

Enter your choice:
```

Screenshot 1: Main menu

```
● ● ● Statistics Calculator

--- Statistics Calculator ---
Enter numbers separated by spaces: 2 4 4 4 5 5 7 9

Results:
Mean: 5.0
Median: 4.5
Mode: [4.0]
Variance: 4.0
Standard Deviation: 2.0
```

Screenshot 2: Statistics - data set 2 4 4 4 5 5 7 9

```
● ● ● Statistics Calculator

--- Statistics Calculator ---
Enter numbers separated by spaces: 3 1 2 6

Results:
Mean: 3.0
Median: 2.5
Mode: No mode
Variance: 3.5
Standard Deviation: 1.8708286933869707
```

Screenshot 3: Statistics - all values unique, so "No mode"

●●● Matrix Calculator - multiplication

```
--- Matrix Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Transpose
5. Determinant
Choose an operation: 3
Enter number of rows for Matrix A: 2
Enter number of columns for Matrix A: 2
Enter Matrix A row by row:
Row 1: 1 2
Row 2: 3 4
Enter number of rows for Matrix B: 2
Enter number of columns for Matrix B: 2
Enter Matrix B row by row:
Row 1: 5 6
Row 2: 7 8

Result:
19 22
43 50
```

Screenshot 4: Matrix multiplication of two 2x2 matrices

●●● Matrix Calculator - determinant

```
--- Matrix Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Transpose
5. Determinant
Choose an operation: 5
Enter number of rows for Matrix: 2
Enter number of columns for Matrix: 2
Enter Matrix row by row:
Row 1: 1 2
Row 2: 3 4

Determinant: -2.0
```

Screenshot 5: Determinant of a 2x2 matrix

●●● Matrix Calculator - validation

```
--- Matrix Calculator ---
1. Addition
2. Subtraction
3. Multiplication
4. Transpose
5. Determinant
Choose an operation: 1
Enter number of rows for Matrix A: 2
Enter number of columns for Matrix A: 2
Enter Matrix A row by row:
Row 1: 1 2
Row 2: 3 4
Enter number of rows for Matrix B: 3
Enter number of columns for Matrix B: 3
Enter Matrix B row by row:
Row 1: 1 2 3
Row 2: 4 5 6
Row 3: 7 8 9
Matrices must have the same dimensions.
```

Screenshot 6: Dimension check when adding a 2x2 and a 3x3 matrix

```
●●● Polynomial Calculator

--- Polynomial Calculator ---
Enter degree of polynomial: 2

Enter coefficients from highest power to constant.
Coefficient of x^2: 3
Coefficient of x^1: 1
Coefficient of x^0: -5

Enter value of x: 2

Polynomial value:
P(2.0) = 9.0
```

Screenshot 7: Evaluating $3x^2 + x - 5$ at $x = 2$

```
●●● Geometry Calculator

--- Geometry Calculator ---
1. Circle
2. Rectangle
3. Triangle
4. Square
5. Sphere
6. Cylinder
Choose a shape: 1
Enter radius: 7
Area: 153.93804002589985
Circumference: 43.982297150257104
```

Screenshot 8: Circle with radius 7

```
●●● Geometry Calculator

--- Geometry Calculator ---
1. Circle
2. Rectangle
3. Triangle
4. Square
5. Sphere
6. Cylinder
Choose a shape: 6
Enter radius: 3
Enter height: 10
Surface Area: 245.04422698000386
Volume: 282.7433388230814
```

Screenshot 9: Cylinder with radius 3 and height 10

```
●●● GCD & LCM Calculator

--- GCD & LCM Calculator ---
Enter first number: 48
Enter second number: 18
GCD: 6
LCM: 144
```

Screenshot 10: GCD and LCM of 48 and 18

11. Testing Approach

Because the program is interactive, it was tested with **validation (black-box) tests**: each module was run with chosen inputs and the printed output was compared with the value obtained by hand calculation or from known formulas. Test cases cover normal values, boundary values (zero, negative numbers, single-row matrices), and invalid situations (incompatible dimensions, non-square determinants, invalid menu choices). To repeat a test, run `python main.py`, select the module and enter the input shown.

Categories tested: normal inputs, boundary inputs, invalid selections and dimension errors.

ID	Module	Input	Expected result	Actual result	Status
T1	Statistics	2 4 4 4 5 5 7 9	Mean 5, median 4.5, mode 4, variance 4, SD 2	Mean 5.0, median 4.5, mode [4.0], variance 4.0, SD 2.0	Pass
T2	Statistics	3 1 2 6	Mean 3, median 2.5, No mode, variance 3.5, SD 1.8708	Same (SD 1.8708286...)	Pass
T3	GCD & LCM	48, 18	GCD 6, LCM 144	GCD 6, LCM 144	Pass
T4	GCD & LCM	0, 7	GCD 7, LCM 0	GCD 7, LCM 0	Pass
T5	GCD & LCM	-12, 18	GCD 6, LCM 36	GCD 6, LCM 36	Pass
T6	Polynomial	$3x^2 + x - 5$, $x = 2$	9	$P(2.0) = 9.0$	Pass
T7	Matrix	$[[1,2],[3,4]] \times [[5,6],[7,8]]$	$[[19,22],[43,50]]$	19 22 / 43 50	Pass
T8	Matrix	Determinant of $[[1,2],[3,4]]$	-2	-2.0	Pass
T9	Matrix	Determinant of 3x3 $[[2,0,1],[1,3,2],[1,1,1]]$	0 (singular matrix)	0.0	Pass
T10	Matrix	Transpose of 2x3 $[[1,2,3],[4,5,6]]$	3x2 matrix $[[1,4],[2,5],[3,6]]$	1 4 / 2 5 / 3 6	Pass
T11	Matrix	Add 2x2 and 3x3	Dimension error message	"Matrices must have the same dimensions."	Pass
T12	Matrix	Determinant of 2x3	Square-matrix error message	"Determinant is only defined for square matrices."	Pass
T13	Matrix	Row with too few values	Re-prompt for the row	"Please enter exactly 2 numbers." then accepted valid row	Pass
T14	Geometry	Circle, $r = 7$	Area 153.938, circumference 43.982	153.93804..., 43.98229...	Pass
T15	Geometry	Cylinder, $r = 3$, $h = 10$	Surface area 245.044, volume 282.743	245.04422..., 282.74333...	Pass
T16	Geometry	Shape choice 9	Invalid choice message	"Invalid choice."	Pass
T17	Main menu	Choice 9, then 6	Invalid message, menu redisplayed, then exit message	"Invalid choice. Please try again.", menu, "Thank you for using the calculator!"	Pass

Result: all 17 validation tests passed. **Known limitation found during testing:** if a non-numeric value such as "abc" is typed where a number is expected, Python raises a `ValueError` and the program stops (observed for the GCD & LCM prompt). Wrapping numeric input in `try/except` is planned as a future

enhancement (Section 14).

12. Challenges Faced

Challenge	How it was addressed
Validating matrix input: users can type too few or too many values in a row, which would break later calculations.	Wrapped row reading in a while loop that re-prompts until the number of values equals the declared column count.
Checking mathematical validity of operations (addition needs equal sizes, multiplication needs $\text{cols}(A) = \text{rows}(B)$, determinant needs a square matrix).	Added explicit dimension checks that print a clear message and return to the menu instead of crashing.
Computing determinants of any size without external libraries.	Implemented recursive cofactor expansion with base cases for 1x1 and 2x2, building each minor with list comprehensions.
Handling special cases in GCD/LCM: zero and negative numbers.	Used <code>abs()</code> in the GCD result and a zero check in the LCM function; tested with 0 and negative inputs.
Mode of a data set: multiple modes or no mode at all.	Used Counter and compared each frequency with the maximum; returns a list of all modes or "No mode" when every value occurs once.
Clean matrix output: floats print as 19.0, 22.0.	Used the <code>:g</code> format specifier so whole numbers print without a trailing .0.
Module naming: the file <code>statistics.py</code> has the same name as Python's built-in statistics module, which can cause confusion or import conflicts.	The program works because the project folder is searched first when running <code>main.py</code> ; renaming it (e.g. <code>stats_calculator.py</code>) is recommended for robustness.
Organising a growing program into clean, independent files with a single entry point.	Split the code into one file per calculator with one public function each and a small <code>main.py</code> controller.

13. Learnings and Key Takeaways

- Breaking a program into **modules with a single responsibility** makes it easier to write, debug and extend.
- Well-known algorithms (Euclid, Horner, cofactor expansion) give correct and efficient solutions when implemented carefully, and their **time complexity** guides where limits exist (e.g. factorial cost of the determinant).
- **Input validation and edge cases** (zero, negative values, empty data, incompatible dimensions) are as important as the main calculation.
- Python features such as list comprehensions, tuple unpacking, `zip`, Counter, f-strings and recursion make mathematical code short and readable.
- Testing with hand-verified examples is an effective way to build confidence, and testing also revealed limitations (non-numeric input) that guide future work.
- Documenting design decisions, diagrams and tests helps others understand the project and helps the author reason about the design.

14. Future Enhancements

- **Robust input handling:** wrap numeric input in try/except with re-prompts so non-numeric text never stops the program; add range checks (e.g. non-negative dimensions).
- **Automated unit tests** using unittest or pytest, which requires separating calculation logic from input/output so functions can be tested directly.
- **Inverse, rank and eigenvalues** for matrices, solving systems of linear equations, and a faster determinant using Gaussian elimination ($O(n^3)$).

- **Polynomial operations** such as addition, multiplication, differentiation, integration and root finding.
- **More statistics:** sample variance, quartiles, correlation and regression; optional charts using matplotlib.
- **More geometry:** cone, triangle from three sides (Heron's formula), and unit conversion.
- **Calculation history and logging** saved to a file, and support for more than two numbers in GCD/LCM.
- **Graphical user interface** (Tkinter) or a web interface, and code clean-up such as removing unused imports in main.py and renaming statistics.py.

15. References

- [1] Python Software Foundation. *Python 3 Documentation*. <https://docs.python.org/3/>
- [2] Python Software Foundation. *math - Mathematical functions*. <https://docs.python.org/3/library/math.html>
- [3] Python Software Foundation. *collections - Container datatypes (Counter)*. <https://docs.python.org/3/library/collections.html>
- [4] Knuth, D. E. *The Art of Computer Programming, Vol. 2: Seminumerical Algorithms* (Euclid's algorithm; Horner's rule).
- [5] Anton, H. and Rorres, C. *Elementary Linear Algebra*. Wiley (matrix operations and determinants by cofactor expansion).
- [6] Fowler, M. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley.
- [7] VITyarthi. *Build Your Own Project - General Project Instructions and Submission Guidelines*.